# Non-Standard Problems

Workshop – Term 3, Week 5

**Susan He, Yoshi He, Austin Song**

# Outline


1. **Housekeeping**
2. **Interactives**
   - Introduction to interactives
   - Cave (IOI 2013)
   - Combo (IOI 2018)
3. **Communication problems**
   - Coins (IOI 2017)
   - Parrots (IOI 2011)
4. **Heuristics**
   - Introduction to partial scoring
   - Initial constructions
   - Iterative strategies
5. **Appendix**

# Attendance!

Please scan the attendance form so you can get free food



# VJudge For Workshop

There is *no* VJudge for today!

(Nobody does the problems anyways, but most of the problems here can be found online)

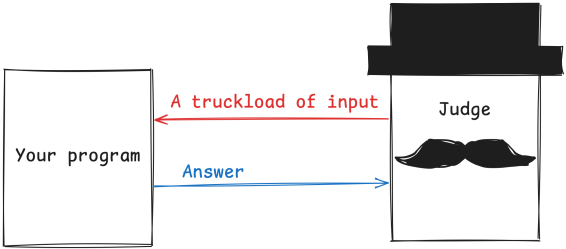
# Outline


1. Housekeeping
2. **Interactives**
   - Introduction to interactives
   - Cave (IOI 2013)
   - Combo (IOI 2018)
3. Communication problems
   - Coins (IOI 2017)
   - Parrots (IOI 2011)
4. Heuristics
   - Introduction to partial scoring
   - Initial constructions
   - Iterative strategies
5. Appendix

# What are interactives?

Hopefully you are all familiar with standard problems

1. Given big block of input in set format
2. Compute and output the answer
3. Receive WA or AC verdict



# What are interactives?

As the name suggests, your program must **interact** with a judge/grader.

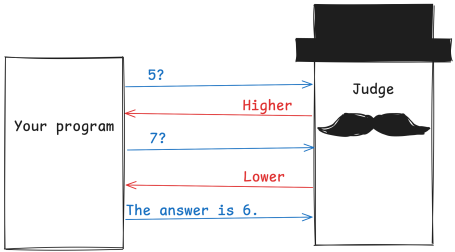



Figure: Two rectangles play a thrilling game of "Guess my number"

1. Given some function(s) to query the grader program
2. Given a template to implement your solution in

# Scoring

Almost all interactive problems score based on the number of queries you use. There are three main types:

1. Problems that directly tell you the target complexity
2. Problems which give exact upper bounds on number of queries
3. Problems that have a hidden limit

For 3rd type, we can usually reverse engineer the limit by submitting dummy solutions.

# Simple example

## Guess the array!

There exists a hidden array $a$ of length $N \geq 3$. You are allowed to make queries get_sum$(i, j)$ where $i \neq j$, and the grader will tell you $a[i] + a[j]$. You must recover the array in at most $N$ queries.

# Simple example

If we can find one of the numbers, then the rest becomes trivial. We can make the following queries:

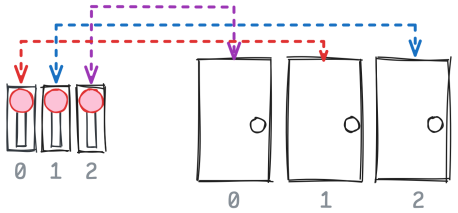
- $q_1 = \text{get_sum}(0,1)$
- $q_2 = \text{get_sum}(1,2)$
- $q_3 = \text{get_sum}(0,2)$

Then, we have $a[0] = \frac{q_1+q_3-q_2}{2}$.

# Cave (IOI 2013)

## Cave

Oh no! You've completely forgotten the password to the security system for your high tech cave! There are $N$ levers numbered from 0 to $N-1$, and $N$ doors numbered from 0 to $N-1$. A lever can be in an **up** or **down** position. The $i$th lever doesn't necessarily connect to the $i$th door, but you remember each lever only connects to exactly one door.



# Cave (IOI 2013)

## Cave

Each lever has a correct state, either up or down. If the state is correct, the door will open, otherwise it will close.

You can try to any combination on the levers with a helpful function `tryCombination(S)`. Here, $S$ is an array of length $N$ indicating the position of each switch. If $S[i] = 0$, then the $i$th switch is up, and $S[i] = 1$ indicates the $i$th switch is down.

Unfortunately, you used opaque metal doors, so you can only see up to the first closed door. `tryCombination(S)` will return the index of the first closed door, or -1 if all doors are open.

# Cave (IOI 2013)

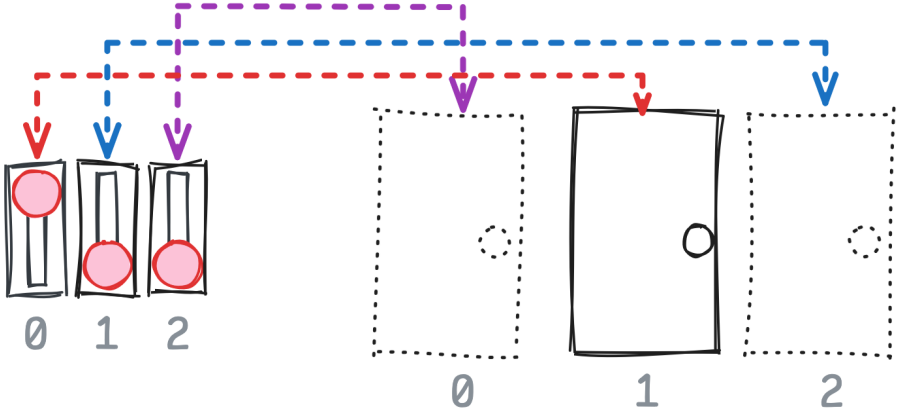



Figure: The correct code is 110. The current configuration is 100. The result of `tryCombination` is 1.

# Cave (IOI 2013)

## Scoring and Constraints

- $1 \leq N \leq 5,000$
- You may call `tryCombination` at most 70,000 times.

Subtasks:

1. Switch $i$ is connect to door $i$.
2. $N \leq 100$.
3. No further constraints.

# Cave (IOI 2013)

## Subtask 1

Switch $i$ is connected to door $i$.

This should be relatively trivial to try to unlock each door one by one.

1. Start with {0,0,0,0...}
2. Is door 1 unlocked? If no, flip lever 1 and query again
3. Is door 2 unlocked? If no, flip lever 2 and query again...

How many queries do we need? At most N queries.

# Cave (IOI 2013)

## Subtask 2

$N \leq 100$.

For each door, you can brute force which lever it is connect to.

1. Start with $\{0,0,0,0...\}$
2. Try changing each lever until the state of door 1 changes.
3. Flip door 1 to the correct position, repeat with all doors after.

How many queries do we need? Each of the 100 doors has 100 possible levers, so this would take 10,000 queries.

# Full Solution

$N \leq 5000$, maximum 70,000 queries.

We can try the same strategy as Subtask 2, except we use binary search instead of trying every switch. For example, with 8 switches, Instead of querying

- $\{1, 0, 0, 0, 0, 0, 0, 0\}$
- $\{0, 1, 0, 0, 0, 0, 0, 0\}$
- and so on...

We can just flip half of the switches with $\{1, 1, 1, 1, 0, 0, 0, 0\}$. Each query narrows down half the search space, and simply repeat until we find the correct lever (of course, we must remember to keep previous doors open).

For each switch, binary search takes $\log(N)$ queries. In total, this would be $N \log(N)$ queries which is adequate for this problem.

# Combo (IOI 2018)

## Combo

You are playing an action video game. The controller has 4 buttons, A, B, X and Y. You can make combo moves, which is any sequence of button presses. There is a **ultra mega super duper uber secret sequence** $S$ made up of button presses, so secret that you only know its length $N$, and that the first character is never repeated.

You can press a sequence of up to $4N$ buttons for a combo move. Let $p$ be the string you press. Your score for $p$ is the length of the **longest prefix** of $S$, which is also a **contiguous substring** of $p$.

$1 \leq N \leq 2000$. Find the secret sequence!

# Combo (IOI 2018)

Example: if the ultra mega super duper uber secret sequence $S$ is ABBYX, you might make the following guesses:

- "BBXY" will receive 0
- "ABXY" will receive 2 for "AB"
- "BYXABB" will receive 3 for "ABB"

Scoring: There is a hard limit of 8,000 queries, but you can get a better score with less. To get full points we can only ask up to N+2 queries.

1. (5 points) $N = 3$
2. (95 points) No additional constraints. For this subtask, your score for each test case is calculated as follows. Let $q$ be the number of calls to `press`.
   - If $q \leq N + 2$, your score is 95.
   - If $N + 2 < q \leq N + 10$, your score is $95 - 3(q - N - 2)$.
   - If $N + 10 < q \leq 2N + 1$, your score is 25.
   - If $\max\{N + 10, 2N + 1\} < q \leq 4N$, your score is 5.
   - Otherwise, your score is 0.

# Brute force?

We can guess each character one by one. For BAXY, we can guess

1. A $=>$ 0
2. B $=>$ 1
3. BA $=>$ 2
4. BAA $=>$ 2
5. BAB $=>$ 2
6. BAX $=>$ 3
7. ... and so on

What is the worst case number of queries required? 4 queries for each character, so 4N.

For the first character, we can determine the character in three queries. For every character after, we can determine the character in two queries. This improves the queries to 2N+1.

# A better way

Brute force: each query eliminates 1 possibility for each character. However, recall that we can input a combo of $4N$ moves for every query.

Let's assume we know what the first character is, and we also know the prefix up to some $i$. We will try to find the next move in the combo.

There are three possible moves. A clue is that we can only use $N + 2$ queries, so each move (except the first) must be found with exactly one query. Can you see how?

# A better way?

If we know the first character is "A", then we only have 3 options. We also know a prefix of the secret up to some i, call this $s$.
pause

Our query can look like

$$\text{s} + \text{"B"} + \text{s} + \text{"XB"} + \text{s} + \text{"XX"} + \text{s "XY"}$$

- If the next move is "B", then the query returns $|\text{s}|+1$.
- If the next move is "X", the query returns $|\text{s}|+2$.
- If the next move is "Y", the query returns $|\text{s}|$.

# Outline


1. Housekeeping
2. Interactives
   - Introduction to interactives
   - Cave (IOI 2013)
   - Combo (IOI 2018)
3. **Communication problems**
   - Coins (IOI 2017)
   - Parrots (IOI 2011)
4. Heuristics
   - Introduction to partial scoring
   - Initial constructions
   - Iterative strategies
5. Appendix

# Communication Problems!

Communication Problems, is kind of like coming up with a secret code for yourself to read later.

- You are given some inputs
- You have a limited amount of, space, operations or resources available to store the information you need about them.
- You then have to complete some tasks that requires you to use the information that you stored.
- By encoding and decoding, you're communicating information to yourself.

# Communication Problems!

In practice, you might be asked to write 2 programs, one to encode, and one to decode. They would not share the same memory space, may both be resource limited in different ways.

Encoder:

- receives some information and some available functions to use such as, `send`, store or `flip_bit`.
- Its job is to encode the information and then store for you to use later.
- is limited in some resource, it might be limited in the amount of times it can call `send`, for example.

Decoder:

- Has to answer some questions correctly.
- Receives information from the encoder. This might mean it receives all available information provided by the encoder in some cases.
- Or it might have some functions available such as `receive` or `read_bit` which might let you read part of the message stored by the encoder. Again you'll be limited in th

# Coins

## Coins

A very bad person (VBP) captured you can your friend and will lock you up forever unless you solve his puzzle!
He presents you with a 8x8 chessboard where each cell has a coin on it, exactly one of which is cursed! Each of the coins is flipped to either heads or tails. He then tells you which of the coins is cursed, you can then flip at least 1 and at most k coins on the board (without marking them, you may flip the same coin multiple times).
VBP then takes the chessboard to your friend (who is in a different room) and asks them to identify the cursed coin. You may communicate your strategy beforehand.
Which coins do you flip?

# Coins

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
| 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 |
| 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 |
| 40 | 41 | 42 | 43 | 44 | 45 | 46 | 47 |
| 48 | 49 | 50 | 51 | 52 | 53 | 54 | 55 |
| 56 | 57 | 58 | 59 | 60 | 61 | 62 | 63 |

# Coins (Subtasks)

1. $c < 2, k = 1$.
2. $c < 3, k = 1$.
3. $k = 64$
4. $k = 8$
5. $k = 1$

We can think of this as the problem of, how do we communicate the position of the cursed coin by flipping only a limited number of coins over?

# Coins: Sub-task 1 and 2

1. $c \leq 1, k = 1$. We need to encode enough information to differentiate 2 options.
   - the coin on cell 0 is cursed
   - or the coin on cell 1 is cursed

We can set/unset the first bit.

2. $c < 3, k = 1$.
   - Similar, but its now 3 options.
   - Ignoring the rest of the board, we focus on the first 3 coins in the top row. No matter how they are set, it's possible to flip at most 1 of them such the other coins are the same parity, and the cursed coin is the other.

# Coins: Sub-task 3 and 4

3. $k = 64$

Similar idea to the previous sub-task. We can set the board so that the cursed coin is the only one facing up. (if the board is already how we need it, we can flip any coin twice).

4. $k = 8$

- We can now flip 8 coins, that doesn't allow us to flip the entire board but it does allow us enough operations to encode some useful information.
- We can write the position of the cursed coin down as a bit string. ($64 = 2^5 < 2^8$)
- Alternatively it's also possible to make it so that only the row and column containing the cursed coin is of even parity.

# Coins: Sub-task 5

$k = 1$
This is much harder harder than the previous parts, 1 is not a big number.

Clearly, one bit worth of information is not to differentiate 64 different options, but can make use of the information rest of the board somehow? This can be useful to us even if we don't have control over what it is like.

An idea that doesn't quite work: Loop over each cell on the board that's flipped to heads. Add to a tally the position of that cell, take mod 64. Once you are done with the whole board, you should be left with a number (mod 64). Now there's a unique number (mod 64) we can add to this total so that the new number by flipping it over from tails to heads.

We want instead of addition some operation where applying the same number again a second time, is the same as not having it done the first time so flipping a cell from tails heads does the same thing as flipping it from heads to tails.

# Coins

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
| 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 |
| 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 |
| 40 | 41 | 42 | 43 | 44 | 45 | 46 | 47 |
| 48 | 49 | 50 | 51 | 52 | 53 | 54 | 55 |
| 56 | 57 | 58 | 59 | 60 | 61 | 62 | 63 |

# Parrots

## Parrots

Yanee's dream is to use her birds to send a message **M** to a land far far away. Her message **M** is a sequence of **N** (not necessarily distinct) integers, each between 0 and 255, inclusive. Yanee keeps **K** specially-trained parrots. All the parrots look the same; Yanee cannot tell them apart. Each bird can remember a single integer between 0 and **R**, inclusive.

Early on, she tried a simple scheme: to send a message, Yanee carefully let the birds out of the cage one by one. Before each bird soared into the air, she taught it a number from the message sequence in order. Eventually, all the birds did arrive at the destination, but they do not necessarily arrive in the order in which they left.

Using this scheme, how can she send send the message so that it is recoverable?

# Parrot: Sub-task 1

Constraints:

- $N = 8$, and each integer in the array M is either 0 or 1.
- Each encoded integer must be in the range from 0 to R=65535, inclusive.
- The number of times you can call the procedure send is at most $K=10\times N$.

There's a few ways to do this. We could just use one parrot and encode the number corresponding to the bit string, or store the index in the 10's digit and the bit in the one's place etc.

# Parrot: Sub-task 2, 3, 4

Constraints:

- $1 \leq N \leq 32$
- Each encoded integer must be in the range from 0 to R=255, inclusive.
- The number of times you can call the procedure send is at most K=10×N.

They're all pretty similar and the question is pretty generous on how many birds we can use, so we'll just jump to part 4 :)

Note that $255 = 2^8 - 1$. We will talk about some of this in terms of the number of bits required.
An issue we needed to solve was to determine the order of the birds.

# Parrot: Sub-task 4

Here are some ways to do this:

We can just store the index/order each bird left in the message each bird carries.
The message is N numbers long, each up to 255. That's $N * 8 = 8N$ bits. We can use up to $10N$ birds. So it would suffice to use $8N$ birds and for each bird to store only 1 bit of information. The other 7 bits can be used to store the position of the bird.

we can also think of the whole message as a bit string of length $N * 8$, for each k from 0 to $8N$, send the number k via a bird if the $k^{th}$ bit is set.

Can we do better?

# Outline


1. Housekeeping
2. Interactives
   - Introduction to interactives
   - Cave (IOI 2013)
   - Combo (IOI 2018)
3. Communication problems
   - Coins (IOI 2017)
   - Parrots (IOI 2011)
4. **Heuristics**
   - Introduction to partial scoring
   - Initial constructions
   - Iterative strategies
5. Appendix

# Real Life

Typically, problems have a very straightforward scoring system:

- There are a set of subtasks, each worth a fixed set of points.
- "Solving" all the test cases in a subtask awards you the points for the subtask.

In real life, problems are not this sanitized.

- e.g. Say you're hired by some logistics company who needs to figure out an optimal route passing through $N = 500$ locations, attempting to minimise some combination of distance, traffic, time, fuel usage, etc.
- You have two approaches:
  1. Solve for an optimal solution via some bloated TSP ($O(N^2 2^N) \approx 10^{155}$ btw).
  2. Guess.

In practice, we tend to do the latter.

# Partial Scoring

A **sliding scale** problem, otherwise known as a problem with **partial scoring**, gives you a percentage score based on *how good your outputted solution is*.

- Remember, problems usually score you on correctness.

This means that often, multiple approaches can all score points!

A subset of these problems are referred to as **heuristic** problems, since you are often solving tasks where exact solutions may be computationally expensive.

You can kind of try whatever, but these problems can usually be broken down into:

1. Preliminary constructions
   - Greedies, DPs, beam search, etc.
   - These usually influence how much you can 'improve' your solution.
2. Iterative metaheuristics
   - Hill climb, simulated annealing, etc.
   - Runtime optimisations are *very* helpful here.

# Bongi (FARIO 2023)

## Bongi

You have $N$ balls, and the $i$th ball has a positive integer $a_i$ written on it. Your job is to divide the balls into $K$ buckets, so that each bucket has at least one ball, and every ball is in one bucket.

A bucket's value is the sum of the ball numbers inside it. Your score is the **geometric mean** of all $K$ bucket values. Maximise your score.

**Note.** The scoring function is

$$\min\left(100 \cdot \frac{10 + S/K - B}{10 + S/K - X}, 100\right)$$

where $X$ is your score, $B$ is the judge score, and $S = \sum_i a_i$.

# Initial Ideas

Geometric mean is a weird scoring criteria, so let's try and get some intuition...

We may be familiar with the AM-GM inequality, which states for $x_i \geq 0$,

$$\frac{x_1 + x_2 + \cdots + x_k}{k} \geq \sqrt[k]{x_1 x_2 \ldots x_k}.$$

This fact holds when $x_i$ is the $i$th bucket's value; and the RHS is what we want to maximise!

- Notice that the LHS is just the average of ball numbers, since all balls must be bucketed. Hence, it's constant!
- So the 'best' we can do is try to increase the RHS to be equal to the LHS.

Intuitively, this happens when buckets are as equal as possible (try subbing $x_i = x_j$ above).

How do we come up with some crude way to make buckets roughly equal?

# A Greedy Algorithm

One possible approach is to go through each ball and randomly place it into a bucket.

- Do we think this is going to do well?

```
Score:  2.62365
```

Maybe a slightly better idea is to place each ball into the current lowest sum bucket.

```
Score:  18.7546
```

Any other ideas?

- We're potentially leaving massive value balls to the very end...
- They really should be motivating our entire assignment.

Let's just process the balls in decreasing order, and apply the greedy.

```
Score:  83.9263
```

## wow

wow they really balanced this problem terribly

# Improvement

Now that we have almost 84 points, maybe we feel more motivated to get to 100.

- Unfortunately, this is where the rubber hits the road :(
- We could design increasingly complex greedy approaches (or even DP solutions), but these almost never improve our score significantly.

Instead, we can rely on iterative improvement algorithms, known as **metaheuristics**.

- These algorithms take an initial state (i.e. a candidate solution), and iteratively (fancy jargon for a `while True`) improve it.
- Typically, the 'closer' a solution is to a global maxima, the more likely we will find it.
  - This is usually why we care about how good our initial constructions are.

The most common of these is the **hill climb**, and its buffed cousin, **simulated annealing**.

# Hill Climb

Let's just forget about this Bongi problem for a moment, and just think about the following problem:

**Array Search**

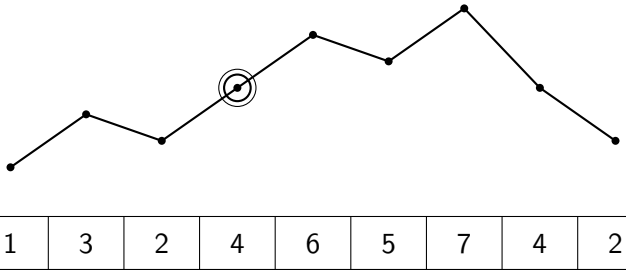
You're given an array of length $N$. Determine the maximum value in the array.

Hopefully we all know at least one way of solving this problem.

- We'll present a bit of a contrived 'solution' for pedagogical purposes.

# Hill Climb (cont.)

Pick a random initial state (i.e. index). We can then visualise our array as follows:




We can **randomly** keep picking an adjacent state to *improve* our solution in.

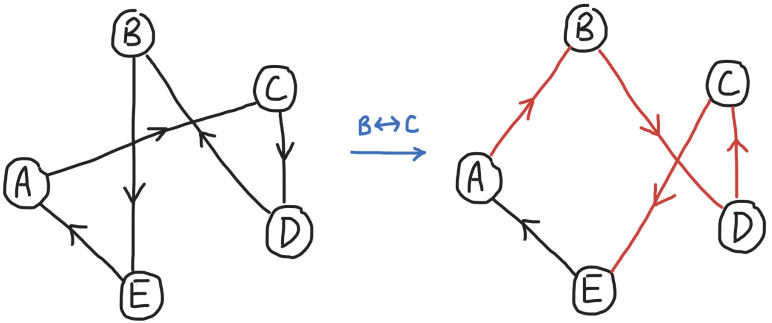
- We'll keep our modification as long as its better than our current state.

This somewhat guarantees an okay solution.

# Hill Climb (cont. cont.)

This is obviously not great for arrays, where we aren't guaranteed any structure/relationship between adjacent elements. But what about problems like TSP (min Hamiltonian cycle)?

- 'Good' cycles are similar to other 'good' cycles in most places.




In this case, 'adjacent' states are one **swap** away! We can also hill climb this :)

# Hill Climb (uncont.)

The **hill climb** metaheuristic consists of the following process:

1. Initialise an arbitrary starting state (the closer to global max the better).
2. For some number of iterations:
   - Perform a randomly selected **transition** (i.e. swap) into an adjacent state.
   - Keep this change if the score improves; otherwise discard it.
3. Return the current state as our best-found solution.

What are the problems with this?

- You could get stuck in a local maxima/minima (depends on what you're optimising).
- Highly RNG-based (need to guess the 'right' transitions to move to the correct area).

So, all in all, this is pretty bad.

# Why?

If hill climb is so bad, why do we use it? There is really only one main reason:

*The state space for complete search is much larger than the length of a hill climb path.*

In general, we are attempting to optimise in a massive state space ($N$-dimensional problems).

- Hill climb paths are roughly on the order of $\sim O(|x|)$ in these spaces.
- Compare this to the $\sim O(|x|^N)$ size of the coordinate space.

Furthermore, if we can optimise the calculation of state score whenever we do a swap (to determine whether the new state is better or not), we can potentially perform hundreds of millions of iterations per second.

- This gives us a better chance of either
  1. finding a pretty good maxima, or
  2. determining stuck-ness a lot quicker, and rerunning with a different initial state.

# Bongi – Hill Climb

What happens when we apply a simple hill climb to Bongi?

- Let's just consider the following transition:

**Transition**

Take a ball from a bucket with $\geq 2$ balls, and move it to another bucket.

We can naively compute the score of each state by looping through buckets in $O(K)$.

How well do we think this will do?

```
Score:  95.6975
```

<0>_<0>

# Two Approaches

We now have two plausible directions for improvement:

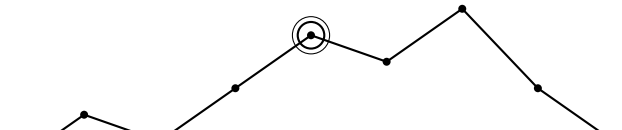
1. Test out different transitions.
   - We can do this if we believe our transitions are too conservative (i.e. don't connect our state space nicely), or too wild (i.e. 'neighbouring' states jump too significantly).
   - We can also try transitions which are easy to optimise, meaning we can iterate a lot more within our time limit.
2. Try fix the local maxima issue.
   - i.e. can we stop getting stuck in local maxima?

Both these approaches in essence are trying to combat the essential weakness of hill climb getting trapped in bad maxima.

We'll come back to the first one, so let's deal with the latter approach.

# Getting Out

Consider a point in time where we might be in a local maxima:



| 1 | 3 | 2 | 4 | 6 | 5 | 7 | 4 | 2 |
|---|---|---|---|---|---|---|---|---|

The easiest way to get out would be to assign a *probability* of leaving – i.e. of taking a worse-scoring state.

- e.g. with 5% probability I will accept transitions to bad states.

What's the issue with this?

# Simulated Annealing

We can think about some things which should matter:

- The worse the score, the less chance we should take it.
  - Hopefully this is pretty obvious...
  - This relies on the fact we've designed our transitions so there's some structure.
- The longer our program goes on, the **less** risky we should be.
  - Presumably, if the heuristic goes well, after many iterations it should be pretty close to global maxima.
  - In this case, we should probably just climb the rest normally.

We can take inspiration from metallurgy! (idk anything abt metallurgy)

- When we want to make a metal strong, we perform an **annealing** process:
  - We first heat up the metal to temporarily rearrange grain structure.
  - Then, we slowly cool the metal to recrystallise.
- As the process goes on, the rate of things changing in the metal decreases.

# Simulated Annealing (cont.)

We can model our probabilities of acceptance with temperature:

- At the beginning, we will jump around quite a bit to try find good states.
- As temperature decreases, we should entertain worse states less.

We can just run a very simple geometric cooling model

$$t \leftarrow t \cdot r$$

for some $0 < r < 1$.

To compute probabilities when transitioning from a state with score $E_o$ to $E_n$ (where $E_o > E_n$), we can utilise the Boltzmann distribution:

$$P \propto \exp\left(-\frac{E_o - E_n}{t}\right)$$

(Note you have to switch $E_n$ and $E_o$ when we minimise.)

# Simulated Annealing (uncont.)

We call this algorithm **simulated annealing**.

Let's apply it to this Bongi problem!

```
Score:  99.7561
```

-_-

You can figure out how to get the remaining 0.24 points.

# Outline


1. Housekeeping
2. Interactives
   - Introduction to interactives
   - Cave (IOI 2013)
   - Combo (IOI 2018)
3. Communication problems
   - Coins (IOI 2017)
   - Parrots (IOI 2011)
4. Heuristics
   - Introduction to partial scoring
   - Initial constructions
   - Iterative strategies
5. **Appendix**

Photo time :)






UNSW CPMSoc